

Two Pointer Templates

Two patterns — pick based on whether pointers converge or march together.

Quick Reference Table

#	Name	Description	Pattern	i init	j init	Loop	Duplicates
167	Two Sum II	Two numbers in sorted array summing to target	Opposite	0	n-1	i < j	No
15	3Sum	All unique triplets summing to 0	Opposite + fix	a+1	n-1	i < j	Skip after hit
18	4Sum	All unique quadruplets summing to target	Opposite + 2 fix	b+1	n-1	i < j	Skip after hit
11	Container With Most Water	Max water between two vertical lines	Opposite	0	n-1	i < j	No
42	Trapping Rain Water	Total trapped water between elevation bars	Opposite	0	n-1	i < j	No
27	Remove Element	Remove all val in-place, return new length	Same dir	0 (write)	0 (read)	j < n	No
26	Remove Duplicates	Each element appears at most once in-place	Same dir	0 (write)	0 (read)	j < n	Compare nums[i-1]
80	Remove Duplicates II	Each element appears at most twice in-place	Same dir	0 (write)	0 (read)	j < n	Compare nums[i-2]
283	Move Zeroes	Move zeroes to end, preserve relative order	Same dir	0 (write)	0 (read)	j < n	No
75	Sort Colors	Sort 0s, 1s, 2s in-place (Dutch Flag)	3-pointer	0 (low)	n-1 (high)	k <= j	No

Pattern 1 — Opposite Two Pointers

```
// MENTAL MODEL: two ends close in on the answer; each comparison rules out one whole end of the range.
// WHY sorted: moving inward only safely eliminates half the space if the array is sorted
int i = 0, j = n - 1;
while (i < j) {
    if (condition == target) {
        /* record */ i++; j--;
    } else if (tooSmall) {
        i++;
    } else {
        j--;
    }
}
```

Sort first. `i` and `j` converge inward. Used when the array is sorted and you can eliminate half the search space each step.

#167 Two Sum II

Description: Sorted array. Find two numbers summing to target. Return 1-based indices.
Key: sum too small → move `i` right. sum too large → move `j` left.
Intuition: Squeeze from both ends; each comparison tells you which side is wrong, so one move eliminates a whole row/column of pairs.

```

class Solution {
    public int[] twoSum(int[] nums, int target) {
        int i = 0, j = nums.length - 1;
        while (i < j) {
            int sum = nums[i] + nums[j];
            if (sum == target) {
                return new int[]{i + 1, j + 1};
            } else if (sum < target) {
                i++;
            } else {
                j--;
            }
        }
        return new int[]{-1, -1};
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#15 3Sum

Description: Find all unique triplets summing to 0. No duplicate triplets in output.

Key: sort first, fix `a`, run opposite two pointers on `[a+1, n-1]`. Skip duplicates at all three levels.

Intuition: Pin one number, then it becomes Two Sum on the rest — turning a triple search into a sweep.

```

class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        Arrays.sort(nums);
        List<List<Integer>> result = new ArrayList<>();
        for (int a = 0; a < nums.length - 2; a++) {
            if (a > 0 && nums[a] == nums[a - 1]) continue; // skip dup a
            int i = a + 1, j = nums.length - 1;
            while (i < j) {
                int sum = nums[a] + nums[i] + nums[j];
                if (sum == 0) {
                    result.add(Arrays.asList(nums[a], nums[i++], nums[j--]));
                    // dup-skip after a hit - see "Duplicate-skip Cheat Sheet" below
                    while (i < j && nums[i] == nums[i - 1]) i++; // skip dup i
                    while (i < j && nums[j] == nums[j + 1]) j--; // skip dup j
                } else if (sum < 0) {
                    i++;
                } else {
                    j--;
                }
            }
        }
        return result;
    }
}

```

Time $O(n^2)$ | **Space** $O(1)$ excluding output ($O(n)$ sort stack)

#18 4Sum

Description: Find all unique quadruplets summing to target.

Key: two nested fix loops + same opposite two pointer inner loop. Cast to `long` — sum of 4 ints can overflow.

Intuition: Pin two numbers, then the inner search is Two Sum — same idea as 3Sum with one more fixed level.

```

class Solution {
    public List<List<Integer>> fourSum(int[] nums, int target) {
        Arrays.sort(nums);
        List<List<Integer>> result = new ArrayList<>();
        int n = nums.length;
        for (int a = 0; a < n - 3; a++) {
            if (a > 0 && nums[a] == nums[a - 1]) continue;
            for (int b = a + 1; b < n - 2; b++) {
                if (b > a + 1 && nums[b] == nums[b - 1]) continue;
                int i = b + 1, j = n - 1;
                while (i < j) {
                    long sum = (long) nums[a] + nums[b] + nums[i] + nums[j];
                    if (sum == target) {
                        result.add(Arrays.asList(nums[a], nums[b], nums[i++], nums[j--]));
                        // dup-skip after a hit – see "Duplicate-skip Cheat Sheet" below
                        while (i < j && nums[i] == nums[i - 1]) i++;
                        while (i < j && nums[j] == nums[j + 1]) j--;
                    } else if (sum < target) {
                        i++;
                    } else {
                        j--;
                    }
                }
            }
        }
        return result;
    }
}

```

Time $O(n^3)$ | **Space** $O(1)$ excluding output ($O(n)$ sort stack)

#11 Container With Most Water

Description: Given heights of vertical lines, find two lines forming the container with most water.

Key: always move the shorter side inward — moving the taller side can only make things worse.

Intuition: Width shrinks every step, so only a taller boundary can ever help; abandon the shorter wall.

```

class Solution {
    public int maxArea(int[] height) {
        int i = 0, j = height.length - 1, max = 0;
        while (i < j) {
            max = Math.max(max, Math.min(height[i], height[j]) * (j - i));
            if (height[i] < height[j]) {
                i++;
            } else {
                j--;
            }
        }
        return max;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#42 Trapping Rain Water

Description: Given elevation heights, compute total water trapped after rain.

Key: water above position $x = \min(\text{maxLeft}, \text{maxRight}) - \text{height}[x]$. Track running max from each side.

Intuition: Water level at a cell is set by the shorter side's tallest wall, so always process from the lower side where that max is already known.

```

class Solution {
    public int trap(int[] height) {
        int i = 0, j = height.length - 1, water = 0;
        int maxI = 0, maxJ = 0;
        while (i < j) {
            if (height[i] <= height[j]) {
                maxI = Math.max(maxI, height[i]);
                water += maxI - height[i];
                i++;
            } else {
                maxJ = Math.max(maxJ, height[j]);
                water += maxJ - height[j];
                j--;
            }
        }
        return water;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

Pattern 2 — Same Direction (Write Pointer)

```

// MENTAL MODEL: a fast reader scans everything; a slow writer trails behind and only copies the keepers.
// WHEN: filter/compact an array in place, no extra buffer
int i = 0; // i = write position
for (int j = 0; j < n; j++) { // j = read position
    if (keepCondition(nums[j]))
        nums[i++] = nums[j];
}
return i;

```

`i` marks where the next valid element goes. `j` scans every element. Only write when the element passes the keep condition.

The generalised duplicate-skip pattern for "at most k duplicates":

```

int i = 0;
for (int j = 0; j < n; j++)
    if (i < k || nums[j] != nums[i - k]) // compare write head to k slots back
        nums[i++] = nums[j];
return i;

```

- $k = 1 \rightarrow$ #26 (no duplicates)
- $k = 2 \rightarrow$ #80 (at most 2)

#27 Remove Element

Description: Remove all occurrences of `val` in-place. Return new length.

Keep condition: `nums[j] != val`

Intuition: The writer only advances on keepers, so it always points at the next free slot for a kept value.

```

class Solution {
    public int removeElement(int[] nums, int val) {
        int i = 0;
        for (int j = 0; j < nums.length; j++)
            if (nums[j] != val) nums[i++] = nums[j];
        return i;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#26 Remove Duplicates from Sorted Array

Description: Remove duplicates in-place so each element appears at most once. Return new length.

Keep condition: `nums[j] != nums[i-1]` — don't write if same as last written.

Intuition: Since the array is sorted, comparing against the last written value is enough to drop every duplicate run.

```
class Solution {
    public int removeDuplicates(int[] nums) {
        int i = 0;
        for (int j = 0; j < nums.length; j++)
            if (i == 0 || nums[j] != nums[i - 1])
                nums[i++] = nums[j];
        return i;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#80 Remove Duplicates from Sorted Array II

Description: Each element may appear at most twice in-place. Return new length.

Keep condition: `nums[j] != nums[i-2]` — don't write if same as two slots back.

Intuition: Looking two slots back lets at most two copies through before the third is rejected.

```
class Solution {
    public int removeDuplicates(int[] nums) {
        int i = 0;
        for (int j = 0; j < nums.length; j++)
            if (i < 2 || nums[j] != nums[i - 2])
                nums[i++] = nums[j];
        return i;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#283 Move Zeroes

Description: Move all zeroes to end while preserving relative order of non-zero elements.

Keep condition: `nums[j] != 0`. Fill tail with zeros after the write pass.

Intuition: Compact the non-zeros to the front in order, then pad the leftover tail with zeros.

```
class Solution {
    public void moveZeroes(int[] nums) {
        int i = 0;
        for (int j = 0; j < nums.length; j++)
            if (nums[j] != 0) nums[i++] = nums[j];
        while (i < nums.length) nums[i++] = 0;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

Pattern 3 — Dutch Flag (3 Pointers)

```
// MENTAL MODEL: one scan partitions into three buckets (low / mid / high) using two boundary pointers.
// WHEN: 3-way partition – sort 0/1/2, segregate by a pivot category
int i = 0, k = 0, j = n - 1;
while (k <= j) {
    if (nums[k] == LO) {
        swap(nums, i++, k++); // confirmed small
    } else if (nums[k] == MID) {
        k++; // confirmed mid, skip
    } else {
        swap(nums, k, j--); // large – don't advance k
    }
}
```

Three regions: `[0, i)` = confirmed small, `[i, k)` = confirmed mid, `(j, n)` = confirmed large. `k` is the frontier.
Do **not** increment `k` after swapping with `j` — the swapped element is unknown.

#75 Sort Colors

Description: Sort array of 0s, 1s, 2s in-place without library sort.
Key: Dutch Flag with LO = 0, MID = 1, HI = 2.
Intuition: A single scan grows a "small" region from the left and a "large" region from the right, leaving mids in the middle.

```
class Solution {
    public void sortColors(int[] nums) {
        int i = 0, k = 0, j = nums.length - 1;
        while (k <= j) {
            if (nums[k] == 0) {
                swap(nums, i++, k++);
            } else if (nums[k] == 1) {
                k++;
            } else {
                swap(nums, k, j--);
            }
        }
    }
    private void swap(int[] a, int x, int y) {
        int t = a[x]; a[x] = a[y]; a[y] = t;
    }
}
```

Time O(n) | Space O(1)

Pattern Comparison

Pattern	Pointers move	Sort needed	Use when
Opposite	<code>i</code> right, <code>j</code> left, converge	Yes (usually)	Sum/target problems on sorted array
Same direction	<code>j</code> scans, <code>i</code> writes	No	Filter / compact array in-place
Dutch Flag	<code>k</code> scans, <code>i</code> low boundary, <code>j</code> high boundary	No	3-way partition (<code><</code> / <code>=</code> / <code>></code>)

Duplicate-skip Cheat Sheet (for Opposite pointers after a hit)

```
// After recording a result at (i, j):
i++; j--;
while (i < j && nums[i] == nums[i - 1]) i++; // skip duplicate i
while (i < j && nums[j] == nums[j + 1]) j--; // skip duplicate j
```

Always check `i < j` inside the skip loop to avoid crossing.